

These five header files will, in turn, include a large number of header files required for the proper working of the ns-3 simulation script. This may not be an optimal solution to our problem of including the required header files alone. But this approach definitely reduces the burden on the ns-3 script writer; instead, the burden is passed on to the system running the simulation with many unnecessary header files included. The next line 'using namespace ns3;' makes every class, function and variable declared inside the ns-3 namespace available for direct usage without using the qualifier 'ns3:'. Remember, we haven't used the standard namespace 'std' of C++. So we need the qualifier 'std::' to get standard C++ constructs, e.g., 'std::cout<< "Hello ns-3";' will work but 'cout<<"Hello ns-3";' will result in an error.

The next line 'NS_LOG_COMPONENT_DEFINE("SimulationOne");' declares a logging component named *SimulationOne*. By referring to this logging object console, message logging can be enabled and disabled. Logging is one of the three methods used in ns-3 to provide simulation data to the user. The other two methods are trace analysis and network topology animation. Even though logging is a very useful way to obtain data from the simulation, the preferred method is trace analysis. With logging, the data is displayed on the terminal. There are seven different log types used in logging. These are: LOG_ERROR, LOG_WARN, LOG_DEBUG, LOG_INFO, LOG_FUNCTION, LOG_LOGIC and LOG_ALL. The amount of information provided increases with each level, with LOG_ERROR providing the least information and LOG_ALL providing the most.

The next line 'Time::SetResolution(Time::NS);' sets the least amount of time by which two events can be separated. The default minimum time is one nanosecond. Now consider the following two lines of code.

```
LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_
INFO);
LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_
INFO);
```

For each log type there is a corresponding log level type, which enables logging of all the levels below it in addition to its level. So the above lines enable logging on the UDP echo client and UDP echo server with log level type LOG_LEVEL_INFO, which will display information from the log types LOG_ERROR, LOG_WARN, LOG_DEBUG and LOG_INFO. If you find this line confusing, don't worry. This line does not affect the topology of the network we are simulating; it only restricts the amount of data given to the user from the terminal. So, if you want, you can use LOG_LEVEL_ALL, which will provide you with all the available information, even that which is unnecessary for your analysis.

From the following line of code onwards, we are dealing with the actual network being simulated. Earlier, we discussed the different abstract network entities used in ns-3 simulation. All these entities are created and maintained with the help of corresponding topology helper classes.

```
NodeContainer nodes;
nodes.Create(2);
```

The class *NodeContainer* in the above code is a topology helper. It helps us deal with the *Node* objects, which are created to simulate real nodes. Here we are creating two nodes by using the *Create* function of the object called *nodes*.

In the following lines of code, a topology helper class called *PointToPointHelper* is called to configure the channel and the network device:

```
PointToPointHelper pointToPoint;
pointToPoint.SetDeviceAttribute("DataRate", StringValue
("10Mbps"));
pointToPoint.SetChannelAttribute("Delay", StringValue
("5ms"));
```

You can observe that the data rate of the network device is set as 10Mbps and the transmission delay of the channel is set as 5ms. If you are unfamiliar with network simulation altogether, then this is how things are done in a simulation. You can specify different values in a simulated network without making any marginal changes. For example, you can change the delay to 1ms or 1000ms. Thus, you can create fundamentally different networks by changing a few lines of code here and there. And that is the beauty of simulation.

The two lines of code shown below will finish configuring the network and the channel. Here, a topology helper called *NetDeviceContainer* is used to create *NetDevice* objects. This is similar to a *NodeContainer* handling *Node* objects.

```
NetDeviceContainer devices;
devices = pointToPoint.Install(nodes);
```

So now, we have nodes and devices configured. Next, we should install protocol stacks on the nodes. The lines 'InternetStackHelper stack;' and 'stack.Install(nodes);' will install the Internet stack on the two nodes. Internet stack will enable protocols like TCP, UDP, IP, etc, on the two nodes.

The next three lines of code shown below deal with node addressing. Each node is given a specific IP address. Those familiar with ns-2 should note the difference. In ns-3, abstract IP addresses used in the simulation are similar to actual IP addresses.

```
Ipv4AddressHelper address;
address.SetBase("192.168.100.0", "255.255.255.128");
Ipv4InterfaceContainer interfaces = address.Assign(devices);
```